

Logaritmanın Özellikleri

$$\log_b^x = y, \quad b^y = x$$

$$\log_b^{(xy)} = \log_b^x + \log_b^y$$

$$\log_b^{x^p} = p \log_b^x$$

$$\log_b^x = \frac{\log_k^x}{\log_k^b}$$

$$\log_b^{(x/y)} = \log_b^x - \log_b^y$$

$$b^{\log_b^x} = x$$

Teğlön Sembolü

$$\sum_{i=a}^b f(i) = f(a) + f(a+1) + \dots + f(b)$$

$$\sum_{i=a}^b f(i) = 0, \quad a > b$$

$$\sum_{i=a}^b c = c \cdot (b - a + 1)$$

$$\sum_{i=a}^b c f(i) = c \sum_{i=a}^b f(i)$$

$$\sum_{i=a}^b (f(i) + g(i)) = \sum_{i=a}^b f(i) + \sum_{i=a}^b g(i)$$

$$\sum_{i=0}^b i = \sum_{i=a-x}^{b-x} i+x, \quad b+x$$

$$\sum_{i=1}^n i^2 = \frac{n \cdot (n+1) \cdot (2n+1)}{6}$$

Asağı ve Yukarı Yuvarlama Fonks.

$$x \in \mathbb{R}, \quad \lfloor x \rfloor = \max \{k \in \mathbb{Z} \mid k \leq x\}$$

$$x \in \mathbb{R}, \quad \lceil x \rceil = \min \{k \in \mathbb{Z} \mid k \geq x\}$$

$$\lfloor x \rfloor \leq x \leq \lceil x \rceil \quad \text{eğer } x \text{ tam sayı ise}$$

$$\lfloor x \rfloor = \lceil x \rceil = x$$

$$\exists x, y \in \mathbb{R}, \lfloor x \rfloor, y \neq \lfloor x, y \rfloor \quad \text{veya} \quad \lceil x \rceil \neq \lceil x, y \rceil$$

$$\forall x \in \mathbb{R}; k, m \in \mathbb{Z}$$

$$\lfloor x-k \rfloor + m = \lfloor x-k+m \rfloor \quad \text{veya}$$

$$\lceil x-k \rceil + m = \lceil x-k+m \rceil$$

$$\lfloor x \rfloor + \lfloor -x \rfloor = \begin{cases} 0, & x \text{ tam sayı ise} \\ -1, & x \text{ tam sayı değilse} \end{cases}$$

$$\sum_{i=1}^n i^2 = \left(\frac{n \cdot (n+1)}{2} \right)^2$$

$$\sum_{i=0}^n r^i = \frac{1-r^{(n+1)}}{1-r}, \quad (r \neq 1)$$

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}, \quad (0 < r < 1)$$

$$\sum_{i=1}^n \frac{1}{i} \approx \ln^n + \gamma \quad \rightarrow \text{Euler-Mascheroni sabiti}$$

$$\sum_{i=1}^{\log n} n = n \log n$$

$$\sum_{i=1}^n \frac{1}{2^i} = 1 - \frac{1}{2^n}$$

$$\sum_{i=0}^{\log n} 2^i = 2^{\log n + 1} - 1 = 2^{n+1} - 1$$

$$\sum_{i=1}^n \frac{1}{2^i} = 2 - \frac{n+2}{2^n}$$

$$\sum_{i=1}^n \frac{i(i+1)}{2} = \frac{n(n+1)(n+2)}{6}$$

$$\sum_{i=1}^n \log i = \log \prod_{i=1}^n i$$

Görpim Sembolü

$$\prod_{i=a}^b f(i) = f(a) \times f(a+1) \times \dots \times f(b)$$

$$\prod_{i=a}^b f(i) = 1 \quad a > b$$

$$\prod_{i=a}^b c = c^{b-a+1}$$

$$\prod_{i=1}^n i = n!$$

$$\prod_{i=1}^n (a_i + b_i) \neq \prod_{i=1}^n a_i + \prod_{i=1}^n b_i$$

$$\prod_{i=1}^n (a_i \times b_i) = \prod_{i=1}^n a_i \times \prod_{i=1}^n b_i$$

$$\log \prod_{i=a}^b f(i) = \sum_{i=a}^b \log f(i)$$



Türev İntegral Aesilik
giriş bilgilerine bakınız.

Döngüler

~~örnek~~ for($i=1; i \leq n; i++$)

++sum;

$$1 + 1, i \leq n \Rightarrow i \leq n-1$$

$$\sum_{i=0}^{n-1} 1 = (n-1) - 0 + 1 = (n-1) + 1 = (n) \rightarrow \text{döngü bu kadar döncek}$$

iterasyon sayısı.

~~örnek~~ for($i=1; i < n; i++$)

++sum;

$$\left. \begin{array}{l} \text{for}(i=1; i < n; i++) \\ ++sum; \end{array} \right\} 1 + 1, i < n \Rightarrow i < n-1$$

$$= \sum_{i=0}^{n-1-1} 1 = \sum_{i=0}^{n-2} 1 = (n-2) - 0 + 1 = (n-2) + 1 = (n-1)$$

~~örnek~~ for($i=4; i < n; i++$)

++sum;

$$\left. \begin{array}{l} \text{for}(i=4; i < n; i++) \\ ++sum; \end{array} \right\} 4 + 1, i < n \Rightarrow i < n-4$$

$$\sum_{i=0}^{n-5} 1 = \underline{\underline{(n-4)}}$$

~~örnek~~ for($i=1; i \leq n; i+=2$)

++sum;

$$1 + 2, i \leq n \Rightarrow i \leq \frac{n-1}{2}$$

$$\sum_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} 1 = \lfloor \frac{n-1}{2} \rfloor - 0 + 1 = \underline{\underline{\lfloor \frac{n+1}{2} \rfloor}}$$

log n = log₂ n Bu derste tobon yazılma "2" dir.

$$\text{for}(i=1; i \leq n; i*=2) \left\{ \begin{array}{l} 1. i \leq n \\ ++sum; \end{array} \right.$$

$$= \log_2 i \leq \log n$$

$$= i \leq \log n$$

$$= \sum_{i=0}^{\lfloor \log n \rfloor} 1 = \lfloor \log n \rfloor - 0 + 1 = \underline{\underline{\lfloor \log n + 1 \rfloor}}$$

$$\text{for}(i=n; i >= 1; i/=2) \left\{ \begin{array}{l} n/2^i >= 1 = n >= 2^i \\ ++sum; \end{array} \right.$$

$$= i \leq \log n$$

$$\sum_{i=0}^{\lfloor \log n \rfloor} 1 = \underline{\underline{\lfloor \log n + 1 \rfloor}}$$

$$\text{for}(i=1; i \leq n; i*=3) \left\{ \begin{array}{l} 1. i \leq n \\ ++sum; \end{array} \right.$$

$$i \leq \log_3 n$$

$$\sum_{i=0}^{\lfloor \log_3 n \rfloor} 1 = \lfloor \log_3 n \rfloor - 0 + 1 = \underline{\underline{\lfloor \log_3 n + 1 \rfloor}}$$

~~Q. 1~~ for (i=10; i<=n+9; i*=3) } 10. $3^i \leq n+9$
++sum;

$$i \leq \log_3 \left(\frac{n+9}{10} \right)$$

$$\lfloor \log_3 \left(\frac{n+9}{10} \right) \rfloor$$

$$\sum_{i=0}^{\lfloor \log_3 \left(\frac{n+9}{10} \right) \rfloor} 1 = \lfloor \log_3 \left(\frac{n+9}{10} \right) \rfloor - 0 + 1$$

$$= \lfloor \log_3 \left(\frac{n+9}{10} \right) + 1 \rfloor$$

~~Q. 2~~ for (i=1; i<=n*n*n; i*=2) } $2^i \leq n^3$
++sum;

$$i \leq 3 \log n$$

$$\lfloor 3 \log n \rfloor$$

$$\sum_{i=0}^{\lfloor 3 \log n \rfloor} 1 = \lfloor 3 \log n \rfloor - 0 + 1$$

$$= \underline{\underline{\lfloor 3 \log n + 1 \rfloor}}$$

~~Q. 3~~ for (i=1; i*i<=n; i++) } $1+i \leq \sqrt{n} = i \leq \sqrt{n}-1$
++sum;

$$\lfloor \sqrt{n}-1 \rfloor$$

$$\sum_{i=0}^{\lfloor \sqrt{n}-1 \rfloor} 1 = \lfloor \sqrt{n}-1 \rfloor + 1 = \underline{\underline{\lfloor \sqrt{n} \rfloor}}$$

~~Q. 4~~ for (i=1; i<=n; i++) } $1+1.i \leq n \quad i \leq n-1$
for (j=1; j<=n; j++) } $1+1.j \leq n \quad j \leq n-1$
++sum;

$$\sum_{i=0}^{(n-1)} \sum_{j=0}^{(n-1)} 1 = \sum_{i=0}^{(n-1)} (n) = \underline{\underline{\frac{(n)^2}{2}}}$$

~~for~~ $\left. \begin{array}{l} \text{for } (i=10; i < n; i++) \\ \text{for } (j=5; j < n-2; j++) \\ \quad ++sum; \end{array} \right\} \begin{array}{l} 10+i < n = \underbrace{i < n-10}_{\lceil n-10 \rceil} \\ 5+j < n-2 = \underbrace{j < n-7}_{\lceil n-8 \rceil} \end{array}$

$$\sum_{i=0}^{\lceil n-11 \rceil} \sum_{j=0}^{\lceil n-8 \rceil} 1 = \underline{\underline{\lceil n-10 \rceil \cdot \lceil n-8 \rceil}}$$

~~for~~ $\left. \begin{array}{l} \text{for } (i=5; i < n; i++) \\ \text{for } (j=10; j < n; j+=2) \\ \quad ++sum; \end{array} \right\} \begin{array}{l} i < n-5 \\ j < \frac{n-10}{2} \end{array}$

$$\sum_{i=0}^{\lceil n-6 \rceil} \sum_{j=0}^{\lceil \frac{n-10}{2} \rceil} 1 = \underline{\underline{\lceil n-5 \rceil \cdot \lceil \frac{n-10}{2} \rceil}} \quad \text{Target Demis}$$

~~for~~ $\left. \begin{array}{l} \text{for } (i=0; i < n; i++) \\ \text{for } (j=0; j < 10; j++) \\ \quad ++sum; \end{array} \right\} \begin{array}{l} i < n \\ j < 10 \end{array}$

$$\sum_{i=0}^{\lceil n-1 \rceil} \sum_{j=0}^9 1 = \sum_{i=0}^{\lceil n-1 \rceil} 10 = \underline{\underline{10 \cdot \lceil n \rceil}}$$

~~for~~ $\left. \begin{array}{l} \text{for } (i=0; i < n; i++) \{ \\ \quad \text{for } (j=1; j < n; j*=2) \\ \quad \quad ++sum; \\ \quad \text{for } (k=0; k < n; k++) \\ \quad \quad ++sum \} \end{array} \right\} \begin{array}{l} i < n, j < \log n, k < n \\ \sum_{i=0}^{\lceil n-1 \rceil} \left(\sum_{j=0}^{\lceil \log n-1 \rceil} 1 + \sum_{k=0}^{\lceil n-1 \rceil} 1 \right) \end{array}$

$$\underline{\underline{\lceil n \rceil (\lceil \log n \rceil + n)}}$$

~~for~~ for($i=0; i \leq n; i++$)

$i \leq n$

for($j=0; j \leq i; j++$)

$j \leq i$

$++sum;$

$$\sum_{i=0}^n \sum_{j=0}^i 1 = \underline{\underline{(n+1) \cdot (n+1)}}$$

for($i=0; i \leq n \times n; i++$)

$i \leq n^2$

for($j=0; j \leq i; j++$)

$j \leq i$

$++sum$

$$\sum_{i=0}^{n^2-1} \sum_{j=0}^i 1 =$$

$$\sum_{i=0}^{n^2-1} (i+1) = \frac{(n^2) \cdot (n^2+1)}{2}$$

~~for~~ for($i=0; i \leq n; i++$)

for($j=0; j \leq i; j++$)

for($k=0; k \leq i; k++$)

$++sum$

$i \leq n, j \leq i, k \leq i$

$$\sum_{i=0}^{n-1} \sum_{j=0}^i \sum_{k=0}^i 1$$

$$\sum_{i=0}^{n-1} \sum_{j=0}^i (i+1)$$

$$\sum_{i=0}^{n-1} (i+1) \cdot (i+1) = \sum_{i=0}^{n-1} (i+1)^2 \rightarrow \sum_{i=1}^n i^2$$

Formula

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$= \frac{(n) \cdot (n+1) \cdot (2n+1)}{6}$$

Recursive Forks

for ($i=1$; $i \leq n$; $i++$)
 $\{ \text{sum} \rightarrow C_3$

$$C_4 + \ln (C_1 + C_2 + C_3)$$

$$1 + 1, i \leq n \Rightarrow i \leq n-1$$

$$T(n) = C_4 + \ln$$

$$S = C_5 \text{ (space)}$$

$$\sum_{i=1}^{n-1} 1 = (n-1+1) = \ln$$

$$S(n) = 2n$$

```
void f(int n) {
    if (n==0) return;
    f(n-1);
}
```

$$T(n) = T(n-1) + C$$

$$T(0) = C$$

$$T(n) = T(n-1) + C$$

$$T(n-1) = T(n-2) + C$$

$$T(n-2) = T(n-3) + C$$

$$T(n-3) = T(n-4) + C$$

$\vdots$ } n iterations

$$+ T(0) = C$$

$$T(n) = C \cdot (n+1)$$

void f(int n) {
 if (n == 1) return;
 f(n/2);
}

$$T(n) = T(n/2) + C$$

$$T(n/2) = T(n/4) + C$$

$$\vdots$$

$$2^i = n \text{ iter.}$$

$$T(1) = C$$

$$T(n) = T(n/2^i) + i \cdot C$$

$$\frac{n}{2^i} = 1 \Rightarrow \log_2 n = i$$

$$T(n) = C(\log_2 n + 1)$$

void f(int n) {
 if (n == 0) return; $\rightarrow C_1$
 f(int i = 0; i < n; i++); $\rightarrow C_2 \cdot n$
 f(n-1);
}

$$0 + 1 + 2 + \dots + (n-1) \Rightarrow i \cdot n$$

$$\sum_{i=0}^{n-1} 1 = n - 1 + 1 = n$$

$$T(n) = T(n-1) + C_2 \cdot n + C_1$$

$$T(0) = C_1$$

$$T(n) = T(n-1) + C_2 n + C_1$$

$$T(n-1) = T(n-2) + C_2(n-1) + C_1$$

$$T(n-2) = T(n-3) + C_2(n-2) + C_1$$

$$\vdots$$

$$n \text{ iter}$$

$$T(0) = C_1$$

$$T(n) = C_2 \left[i \cdot n \left(\frac{i \cdot (i+1)}{2} \right) \right] + (i+1) C_1$$

$$T(n) = C_2 \left(\frac{n(n+1)}{2} \right) + (n+1) C_1$$

Call's

void f(int n) {

if (n == 0) return; $\rightarrow C_1$

for (int i = 0; i < n; i++) $\rightarrow C_2 \cdot n + (1) = C_1 + C_2 \cdot n + C_1$

f(n/2);

}

$$T(n) = T(n/2) + C_1 + C_2 \cdot n$$

$$T(0) = C_1$$

$$0 + 1 \cdot i < n \rightarrow i < n$$

$$\sum_{i=0}^{n-1} 1 = (n-1+1) = n$$

$$T(n/2) = T(n/4) + C_1 + C_2 \cdot n/2$$

$$T(n) = T(n/4) + C_1 + C_2 \cdot n/2 + C_2 \cdot n + C_1$$

$$T(n) = T(n/4) + 2C_1 + C_2 \cdot n \left(\frac{1}{2} + 1 \right)$$

$$T(n/2) = T(n/4) + C_1 + C_2 \cdot n/2$$

$$T(n) = T(n/8) + 3C_1 + C_2 \cdot n \left(\frac{1}{4} + \frac{1}{2} + 1 \right)$$

$$T(n) = T\left(\frac{n}{2^i}\right) + iC_1 + C_2 \cdot n \left(\frac{1}{2^0} + \frac{1}{2^1} + \frac{1}{2^2} + \dots + \frac{1}{2^{i-1}} \right)$$

$$\frac{n}{2^i} = 1 \Rightarrow \log n = i$$

$$T(n) = T(1) + \log n C_1 + C_2 \cdot n \left(\frac{1}{2^0} + \dots + \frac{1}{2^{\log n - 1}} \right)$$

$$T(n) = C_1(2 + \log n) + C_2(2n - 1)$$

~~void f(int n) {~~

~~if (n==0) return; → C~~

~~f(n/2);~~

~~f(n/2);~~

~~}~~

$$T(n/2) = 2T(n/4) + C$$

$$= 2T(n/4) + C$$

$$T(n) = 2[2T(n/4) + C] + C$$

$$= 2^2 T(n/4) + 2C + C$$

$$T(n) = 2T(n/2) + C$$

$$T(0) = C$$

$$T(1) = 3C$$

$$T(n/2) = 2T(n/4) + C$$

$$= 2T(n/4) + C$$

$$T(n) = 2^3 T(n/8) + 2^2 C + 2^1 C + 2^0 C$$

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + C(2^0 + 2^1 + \dots + 2^{i-1})$$

$$\frac{n}{2^i} = 1 \Rightarrow \log n = i$$

$$T(n) = 2^{\log n} T(1) + C(2^0 + 2^1 + \dots + 2^{\log n - 1})$$

$$T(n) = 2^{\log n} \cdot 3C + C \cdot (n-1)$$

$$T(n) = 3cn + cn - C$$

$$\underline{\underline{T(n) = 4cn - C}}$$


```
void f(int n) {
```

```
    if (n == 0) return; → C
```

```
    f(n-2);
```

```
    f(n-1);
```

```
}
```

$$T(n) = 2T(n-1) + C$$

$$T(0) = C$$

$$n-i = 0 \Rightarrow n = i$$

$$T(n-1) = 2T(n-2) + C$$

$$T(n) = 2[2T(n-2) + C] + C$$

$$T(n) = 2^2 T(n-2) + 2C + C$$

$$T(n-2) = 2T(n-3) + C$$

$$T(n) = 2^2 [2T(n-3) + C] + 2C + C$$

$$T(n) = 2^3 T(n-3) + 2^2 C + 2C + C$$

$$T(n) = 2^i T(n-i) + C(2^0 + \dots + 2^{i-1})$$

$$T(n) = 2^n T(0) + C(2^0 + \dots + 2^{n-1})$$

$$T(n) = 2^n \cdot C + C(2^n - 1)$$

$$T(n) = C(2^{n+1} - 1)$$

$$T(n) = c(2^n - 1)$$

Master Theorem

$$\begin{aligned} T(n) &= a \cdot T(n/b) + f(n) \\ &= a \cdot T(n/b) + O(n^c) \end{aligned}$$

1) Eger $c < \log_b a$ ise

$$T(n) = O(n^{\log_b a})$$

2) Eger $c = \log_b a$ ise

$$T(n) = O(n^c \log n)$$

3) Eger $c > \log_b a$ ise

$$T(n) = O(n^c)$$

fibonacci sorusunun
kolayca formülünü

Akra - Bazzi Yöntemi

$$T(n) = f(n) + \sum_{i=1}^k a_i T(n/b_i)$$

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{f(x)}{x^{p+1}} dx\right)\right)$$

$$\sum_{i=1}^k a_i (b_i)^p = 1$$

$$T(n) = \underbrace{1}_{a_1} \cdot \underbrace{1}_{b_1} \cdot \underbrace{1}_{a_2} \cdot \underbrace{1}_{b_2} \cdot T(n/2) + T(n/3) + n$$

$$\sum_{i=1}^{k=2} a_i(b_i)^p = 1 \Rightarrow 1 \cdot \left(\frac{1}{2}\right)^p + 1 \cdot \left(\frac{1}{3}\right)^p = 1$$

$$\left(\frac{1}{2}\right)^p + \left(\frac{1}{3}\right)^p = 1$$

$$\int_1^n \frac{f(x)}{x^{p+1}} dx = \int_1^n \frac{n}{x^{p+1}} dx = n \int_1^n \frac{1}{x^{p+1}} dx \Rightarrow \boxed{\int x^a dx = \frac{x^{a+1}}{a+1}, a \neq -1}$$

$$= n \int_1^n x^{-(p+1)} dx$$

$$= n \left[\frac{x^{-p-1}}{-p-1} \right]_1^n$$

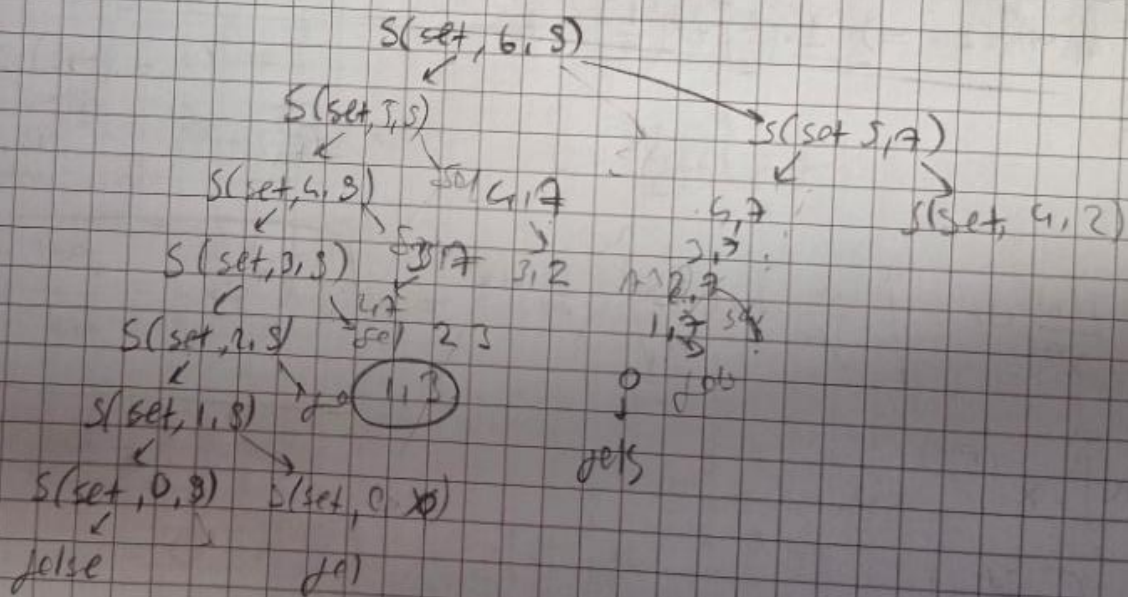
$$= \frac{n(n^{-p-1} - 1)}{-p-1}$$

$$\Theta(p^p(1 + \dots))$$

$$= n^p + p^p \left(\frac{n(n^{-p-1} - 1)}{-p-1} \right)$$

$$= p^p + \frac{n-1}{p}$$

```
public static void main (String [] args) {  
    int set[] = {3, 30, 4, 12, 5, 23};  
    int n = 6;  
    int target = 9;  
    if (subsetSum (set, n, target))  
        cout << "Subnet exist";  
    else  
        cout << "Subnet does not exist";  
}
```


$$S(\text{set}, 3, 3)$$

es ist dann: $T(n) = O(1)$

$$T(n) = c$$

in letzter durch: $T(n) = 2T(n-1) + c \Rightarrow T(n) = c(2^{n+1} - 1)$
 $T(0) = c$

$$T(0) = C$$
$$O(2^n)$$

$$T(n) = 2T(n-1) + C$$

$$T(0) = C$$

$$T(n-1) = 2T(n-2) + C$$

$$T(n) = 2 \cdot [2T(n-2) + C] + C$$

$$T(n) = 2^2 T(n-2) + 2C + C$$

$$\sum_{i=0}^n r^i = \frac{1-r^{(n+1)}}{1-r} (r+2)$$

$$\sum_{i=0}^n 2^{i^0} = 2^{n+1} - 1 = O(2^n) \quad ? \quad \underline{\text{Cetis}}$$

$$P_n + \underbrace{P + \dots + P}_p = 1$$

$$p = \frac{1-P_n}{n}$$

$$T(n) = (2^{(n+1)} - 2) P_n + \sum_{i=0}^{2^{n+1}-2} (i+1) \cdot P_i$$

Break 2 keys

$$T(n) = (2^{n+1} - 2)P_n + \frac{(2^{n+1} - 2)(2^{n+1} - 1)}{2} \cdot \frac{(1 - P_n)}{n}$$

$$= (2^{n+1} - 2)P_n + \frac{(2^{n+1} - 2)(1 - P_n)}{2}$$

$$0 \leq P_n \leq 1$$

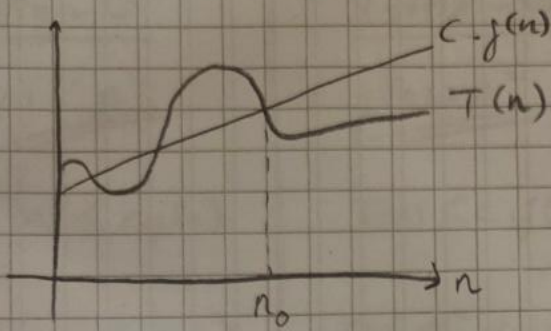
$$\frac{2^{n+1} - 2}{2} \leq T(n) \leq \frac{2^{n+1} - 2}{2}$$

$$T(n) \in O(2^n)$$

Big-Oh Notasyonu

$T(n) \in O(f(n))$ eğer $\exists c > 0$ ve $\exists n_0 > 0$

öyle ki $0 \leq T(n) \leq c \cdot f(n)$ tüm $n \geq n_0$



Little-Oh Notasyonu

$T(n) \in o(f(n))$ eğer $\forall c > 0$ ve $\exists n_0 > 0$

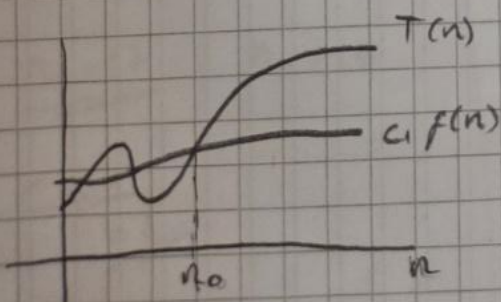
öyle ki $0 \leq T(n) < c \cdot f(n)$ tüm $n \geq n_0$

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$$

Big-Omega Notasyonu

$T(n) \in \Omega(f(n))$ eğer $\exists c > 0$ ve $\exists n_0 > 0$

öyle ki $T(n) \geq c \cdot f(n) \geq 0$ tüm $n \geq n_0$



Little-Omega Notasyonu

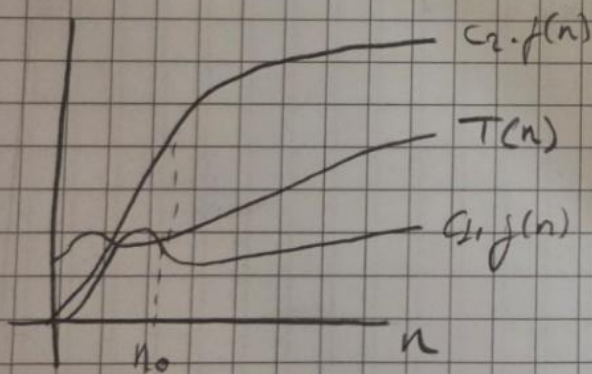
$T(n) \in \omega(f(n))$ eğer $\forall c > 0$ ve $\exists n_0 > 0$
öyle ki $T(n) > c \cdot f(n) > 0$ tüm $n \geq n_0$

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = \infty$$

Big-Theta Notasyonu

$T(n) \in \Theta(f(n))$ eğer $\exists c_1 > 0, \exists c_2 > 0$ ve $\exists n_0 > 0$

öyle ki $0 \leq c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ tüm $n \geq n_0$



$$\begin{aligned} T(n) &= n \log n + n \\ &\Downarrow \\ f(n) &= n \log n \end{aligned}$$

Def $T(n) = \frac{n^2}{4} - 4n \quad n \in \mathbb{Z}^+$ Big-O, Little-O, Big-Omega, Little-Omega

Big-Theta notasyonları için $c_1, c_2, n_0, \epsilon, f(n)$ değerlerini bulun.

$T(n)$ 'i her bir notasyona göre ifade edin.

Big-O

$f(n) = n^2$

$T(n) \leq c_1 f(n)$

$0 \leq \frac{n^2}{4} - 4n \leq c_1 n^2$

$0 \leq \frac{1}{4} - \frac{4}{n} \leq c_1$

$n_0 = 17 \quad n_0 > 16$

$c_1 = \frac{1}{68}$

Little-O

$f(n) = n^2$

$T(n) < c_1 f(n)$

$0 \leq \frac{n^2}{4} - 4n < c_1 n^2$

$0 \leq \frac{1}{4} - \frac{4}{n} < c_1$

$n_0 = 16 \quad 0 < c_1$

$n \geq n_0 = 16$

Big-Omega

$f(n) = n^2$

$T(n) \geq c_1 f(n) \geq 0$

$\frac{1}{4} - \frac{4}{n} \geq c_1 \geq 0$

$n_0 = 17 \quad n_0 > 16$

$c_1 = \frac{1}{68}$

Little-Omega

$f(n) = n^2$

$T(n) > c_1 f(n) > 0$

$\frac{1}{4} - \frac{4}{n} > c_1 \geq 0$

$n_0 = 17 \quad n_0 > 16$

$c_1 = \frac{1}{68}$

Big-Theta

$f(n) = n^2$

$0 < c_1 f(n) \leq T(n) \leq c_2 f(n)$

$0 < c_1 \leq \frac{1}{4} - \frac{4}{n} \leq c_2$

$n_0 = 17 \quad n_0 > 16$

$0 < c_1 \leq \frac{1}{68}$

$c_2 \leq \frac{1}{68}$

Kuralları Birleştirme

- 1) $f(n) \in O(g(n))$ ve $g(n) \in O(h(n))$ ise
o zaman $f(n) \in O(h(n))$
- 2) herhangi bir $k > 0$ için, $f(n) \in O(k \cdot g(n))$ ise
o zaman $f(n) \in O(g(n))$
- 3) $f_1(n) \in O(g_1(n))$ ve $f_2(n) \in O(g_2(n))$ ise
o zaman $f_1(n) + f_2(n) \in O(\max(g_1(n), g_2(n)))$
- 4) $f_1(n) \in O(g_1(n))$ ve $f_2(n) \in O(g_2(n))$ ise
o zaman $f_1(n) \cdot f_2(n) \in O(g_1(n) \cdot g_2(n))$

Fonksiyon Kuyasma

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

- 1) Limit ∞ 'a gidiyorsa $f(n)$ daha hızlı büyüyordur.
bu durumda $f(n), \Omega(g(n))$ 'dedir denir
- 2) Limit 0'a gidiyorsa $g(n)$ daha hızlı büyüyordur.
bu durumda $f(n), O(g(n))$ 'dedir denir
- 3) Limit 0'dan farklı bir sabite gidiyorsa iki fonksiyonda aynı oranda büyüyordur. o zaman $f(n) \in \Theta(g(n))$ 'dir.

$$f(n) = 2n \log n$$

$$g(n) = n^2$$

$$\lim_{n \rightarrow \infty} \frac{2n \log n}{n^2} = \lim_{n \rightarrow \infty} \frac{2 \log n}{n} = \frac{\infty}{\infty}$$

$$\text{art} \Rightarrow n^2) n \log n$$

$$\lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)} = \frac{\frac{2}{n \ln 2}}{1} = \frac{2}{n \ln 2} = \frac{2}{\infty} = 0$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

$$\underline{f(n) \in O(g(n))}$$

semleges lehet

Arányos és Súlyozott Alg. inciderencia

Insertion Sort

```
void Insertion-Sort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        for (j = i - 1; j >= 0 && arr[j] > key; j--)
            arr[j + 1] = arr[j];
        arr[j + 1] = key;
    }
}
```

5, 4, 2, 1, 6, 3
 ↑ ↑
 j i
 5 5 2 1 6 3
 ↑ ↑
 j i
 4 5 2 1 6 3
 4 4 5 1 6 3
 2 4 5 1 6 3

$$1 + 1, i < n$$

$$i - 1 - j \geq 0$$

$$i < n - 1$$

$$j \leq i - 1$$

$$\sum_{i=0}^{n-2} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-2} i = \frac{n^2 - 3n + 2}{2}$$

$T(n)$

$$f(n) = n^2$$

$$0 \leq c_1 f(n) \leq T(n) \leq c_2 f(n)$$

$$\exists c_1 > 0, \exists c_2 > 0, \exists n_0 > 0$$

$$n \geq n_0$$

$$c_1 n^2 \leq \frac{n^2 - 3n + 2}{2} \leq c_2 n^2$$

$$c_1 \leq \frac{1}{2} - \frac{3}{2n} + \frac{2}{2n^2} \leq c_2$$

$$n_0 = 3$$

$$c_1 \leq \frac{1}{9} \leq c_2$$

$$c_1 = c_2 = \frac{1}{9}$$

$$T(n) = \Theta(n^2)$$

Bubble Sort

```
void bubble_sort(int arr[], int n) {
```

```
    int temp;
```

```
    bool swapped;
```

```
    for (int i = 0; i < n-1; i++) {
```

```
        swapped = false;
```

```
        for (int j = 0; j < n-i-1; j++) {
```

```
            if (arr[j] > arr[j+1]) {
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j+1];
```

```
                arr[j+1] = temp;
```

```
                swapped = true;
```

```
            }
```

```
        }
```

```
        if (!swapped)
```

```
            break;
```

```
    }
```

$$0 + 1 + \dots + (n-1)$$

$$0 + 1 + \dots + (n-i-1)$$

$$1 + \dots + (n-1)$$

$$1 + \dots + (n-i-1)$$

$$\sum_{i=0}^{n-2} \sum_{j=0}^{n-i-2} 1 = \sum_{i=0}^{n-2} (n-i-1)$$

$$(n-2) + (n-3) + \dots + 1$$

$$= \frac{(n-2)(n-1)}{2} = \frac{n^2 - n}{2} \quad T(n)$$

$$c_1 \cdot n^2 \leq \frac{n^2 - n}{2} \leq c_2 \cdot n^2$$

$$c_1 \leq \frac{1}{2} - \frac{1}{2n} \leq c_2$$

$$n_0 = 2$$

$$c_1 = c_2 = \frac{1}{4}$$

$$T(n) = \Theta(n^2)$$

Quick Sort

```
void qsort (int arr[], int n) {
```

```
    d-q (arr, 0, n-1, n);
```

```
void d-q (int arr[], int low, int high, int n) {
```

```
    if (low < high) {
```

```
        int pivot = partition (arr, low, high, n);
```

```
        d-q (arr, low, pivot-1, n);
```

```
        d-q (arr, pivot+1, high, n);
```

$$T(n) = T(n-1) + c \cdot n$$

$$T(n) = c$$

$$T(n-1) = T(n-2) + c(n-1)$$

$$T(n) = T(n-2) + c(n-1) + cn$$

$$T(n-2) = T(n-3) + c(n-2)$$

$$T(n) = T(n-3) + c(n-2) + c(n-1) + cn$$

$$T(n) = T(n-3) + c[(n-2) + (n-1) + n]$$

$$T(n) = T(n-i) + c[(n-i+1) + \dots + n]$$

$\begin{matrix} n-i+1 \\ i=n-1 \end{matrix}$

$$T(n) = T(1) + c[2 + 3 + \dots + n]$$

$$T(n) = c[1 + 2 + \dots + n] \Rightarrow T(n) = \frac{n(n+1)}{2} = \frac{n^2 + n}{2}$$

$$T(n) = \Theta(n^2)$$

```
int partition (int arr[], int low, int high) {
```

```
    ...
```

```
    for (int j = low; j < high; j++) {
```

```
        ...
```

```
    }
```

```
    ...
```

```
}
```


Linear Search

SIRASIZ VERTLERDE

```
int linearSearch(int arr[], int n, int item) {
```

```
    for(int i=0; i<n; i++)
        if(arr[i] == item) return i;
```

```
    return -1;
```

```
}
```

$$p_1 + \underbrace{p_2 + \dots + p_n}_{=1} = 2$$

$$p_1 \cdot n + p_n = 1$$

$$p = \frac{1-p_n}{n}$$

$$T(n) = n p_n + \sum_{i=0}^{n-1} (i+1) \cdot p_i$$

$$T(n) = n p_n + \left(\frac{n \cdot (n+1)}{2} \right) p_i \rightarrow \frac{1-p_n}{n}$$

$$T(n) = n p_n + \frac{(n+1)(1-p_n)}{2}$$

$$T(n) = \frac{n p_n - p_n + n + 1}{2}$$

$$0 \leq p_n \leq 1$$

$$T(n) = \frac{n+1}{2}$$

$$T(1) = n$$

$$\frac{n+1}{2} \leq T(n) \leq n$$

$$c_1 f(n) \leq T(n) \leq c_2 f(n)$$

$$f(n) = n$$

$$c_1 \cdot n \leq \frac{n+1}{2} \leq c_2 \cdot n$$

$$c_1 \leq \frac{1}{2} + \frac{1}{2n} \leq c_2$$

$$\begin{matrix} c_0 = 1 \\ c_1 = c_2 = 1 \end{matrix}$$

$$T(n) = \Theta(n)$$

Jump Search

Sirali Arana

```
int JumpSearch (int arr[], int n, int item) {
```

```
    int j = sqrt(n);
    int lowerbound = 0;
    int upperbound = j;
```

```
    while (arr[upperbound] <= item && upperbound < n) {
```

```
        lowerbound = upperbound;
        upperbound += j;
```

```
        if (upperbound > n-1) {
            upperbound = n;
            break;
        }
```

```
    }
```

```
    for (int i = lowerbound; i < upperbound; i++)
```

```
        if (arr[i] == item)
            return i;
```

```
    return -1;
```

```
}
```

1 3 12 21 43 57 61 82

arr[0:j]

arr[j:j]

arr[m:j]

$$m, j \leq n \leq (m+1) \cdot j$$

$$m \leq \frac{n}{j}$$

$$T(n) = \left\lfloor \frac{n}{j} \right\rfloor + (j-1)$$

$$\left(\frac{n}{j} + j - 1 \right)' = 0$$

$$-\frac{n}{j^2} + 1 = 0$$

$$\left(-\frac{n}{j^2} + 1 \right)' = 0$$

$$\boxed{n=j^2 \quad j=\sqrt{n}}$$

$$\frac{2n}{j^3} = f(n)$$

$$T(n) = \left\lfloor \frac{n}{\sqrt{n}} \right\rfloor + \sqrt{n} - 1$$

$$= \left\lfloor \sqrt{n} \right\rfloor + \sqrt{n} - 1$$

$$\approx 2\sqrt{n} - 1$$

$$c_1 \sqrt{n} \leq 2\sqrt{n} - 1 \leq c_2 \sqrt{n}$$

$$c_1 \leq 2 - \frac{1}{\sqrt{n}} \leq c_2$$

$$T(n) = \Theta(\sqrt{n})$$

$$\underline{c_1 = 1}$$

$$\underline{c_2 = 2}$$

$$f(n) = \frac{2n}{(\sqrt{n})^3}$$

$$f(n) = \frac{2}{\sqrt{n}} > 0$$

Binary Search

Srdi Veri

```
int bs-iterative(int arr[], int n, int target){
```

```
    int left = 0, right = n-1;
```

```
    while (left <= right){
```

```
        int mid = left + (right - left) / 2;
```

```
        if (arr[mid] == target) return mid;
```

```
        else if (arr[mid] < target) left = mid + 1;
```

```
        else right = mid - 1;
```

```
    }
```

```
    return -1;
```

```
int bs-recursive(int arr[], int left, int right, int target){
```

```
    if (left > right) return -1;
```

```
    int mid = left + (right - left) / 2;
```

```
    if (arr[mid] == target) return mid;
```

```
    else if (arr[mid] < target) return bs-recursive(arr, mid+1, right, target);
```

```
    else return bs-recursive(arr, left, mid-1, target);
```

```
}
```

$$T(n) = T(n/2) + c \quad T(1) = c$$

$$c_1 \log n \leq \log n + 1 \leq c_2 \log n$$

$$T(n) = T(n/2) + c$$

$$n/2^i = 1 \Rightarrow i = \log n$$

$$T(n) = \log n + c$$

$$c_1 \leq 1 + \frac{1}{\log n} \leq c_2$$

$$T(n) = c(\log n + 1)$$

$$M_0 = 2$$

$$c_1 = 3 \quad c_2 = 3$$

$$f(n) = \log n$$

$$T(n) = \Theta(\log n)$$

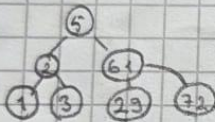
Binary Search Tree

```
private class Node {
    public int key;
    public Node left;
    public Node right;
    public Node(int key) {
        this.key = key;
        left = right = null;
    }
}
```

// Recursive Search

```
Node search (Node root, int key) {
    if (root != null || root.key == key)
        return root;
    if (key < root.key)
        return search (root.left, key);
    return search (root.right, key);
}
```

Search (root, 29):



En Kütçe durum

$$T(n) = T(n-1) + c$$

$$T(1) = c \quad c = O(1)$$

Ortalama durum

$$T(n) = T(n/2) + c$$

$$\log_2 1 = 0 \quad T(n) = c \cdot (\log n + 1)$$

$$f(n) = n^0$$

$$O(n^0 \log n) = O(\log n)$$

$$T(n) = T(n-i) + i \cdot c$$

$$n-i = 1$$

$$i = n-1$$

$$T(n) = (n-1) \cdot c$$

$$= c + (n-1) \cdot c$$

$$O(n)$$

$T(n) = (\text{Her bir seviyede: değim sayısı} \times \text{Derinlik}) / \text{Toplam Değim sayısı}$

$$= \frac{2^0 \cdot \log 2^0 + 2^1 \cdot \log 2^1 + 2^2 \cdot \log 2^2 + \dots + 2^{\log n} \cdot \log 2^{\log n}}{2^0 + 2^1 + 2^2 + \dots + 2^{\log n}}$$

$$\hookrightarrow \sum_{i=0}^{\log n} 2^i = 2^{\log n + 1} - 1 = 2^{\log n} \cdot 2 - 1 = n \cdot 2 - 1 = 2n - 1$$

Bazı

$$= 2^0 \cdot \log 2^0 + 2^1 \cdot \log 2^1 + \dots + 2^{\log n} \cdot \log 2^{\log n} / 2n - 1$$

$$= 1/2n - 1 \cdot \sum_{i=0}^{\log n} i \cdot 2^i = 1/2n - 1 \cdot \sum_{i=0}^{\log n} \left(\sum_{j=0}^i 1 \right) \cdot 2^i$$

$$= 1/2n - 1 \cdot \sum_{i=0}^{\log n} \sum_{j=0}^i 2^i = 1/2n - 1 \cdot \sum_{i=0}^{\log n} \sum_{j=0}^i 2^i = \sum_{i=0}^{\log n} \sum_{j=0}^i 2^i$$

$$= 1/2n - 1 \cdot \sum_{j=0}^{\log n} \sum_{i=j}^{\log n} 2^i = 1/2n - 1 \cdot \sum_{j=0}^{\log n} \sum_{i=0}^{\log n - j} 2^{i+j} = 1/2n - 1 \cdot \sum_{j=0}^{\log n} \sum_{i=0}^{\log n - j} 2^j \cdot 2^i$$

$$= 1/2n - 1 \cdot \sum_{j=0}^{\log n} 2^j \sum_{i=0}^{\log n - j} 2^i = 1/2n - 1 \cdot \sum_{j=0}^{\log n} 2^j \cdot 2^{\log n - j + 1} - 1$$

$$= 1/2n - 1 \cdot \sum_{j=0}^{\log n} 2^{\log n + 1} - 2^j = 1/2n - 1 \cdot \left(\sum_{j=0}^{\log n} 2^{\log n + 1} - \sum_{j=0}^{\log n} 2^j \right)$$

$$= 1/2n - 1 \cdot (2^{\log n + 1} \cdot \log n - (2^{\log n + 1} - 1 - 1))$$

$$= 1/2n - 1 \cdot (2n \cdot \log n - (2n - 2))$$

$$= 1/2n - 1 \cdot (2n \log n - 2n + 2) \approx 1/2n - 1 \cdot (2n \log n - 2n + 2)$$

$$\approx \log n - 1 + \frac{1}{n}$$

Iterative Insertion

```
Node insert(Node root, int key) {
    Node newNode = new Node(key);
    if(!root) return newNode;
    Node current = root;
    Node parent = null;
    while(current) {
        parent = current;
        if(key < current.key)
            current = current.left;
        else
            current = current.right;
    }
    if(key < parent.key)
        parent.left = newNode;
    else
        parent.right = newNode;
    return root;
}
```

En kiti durum:
 $n, n-1, n-2, \dots, 0$
 $i-n=0 \quad n=i$

Ortalama durum:
 $n, n/2, n/4, \dots, 1$

$n/2 = 1 \quad n = 2^i$

$\log n = i$

$T(n) = T(n-1) + O(1)$

$T(n) = T(n/2) + O(1)$

Ekleme, Silme, Arama ve Güncelleme
 zamanları

kek $\rightarrow O(n)$

Orta $\rightarrow O(\log n)$

izi $\rightarrow O(1)$

```
void inorder(Node root) {
    if(root != null) {
        inorder(root.left);
        System.out.println(root.key);
        inorder(root.right);
    }
}
```

```
void preorder(Node root) {
    if(root != null) {
        System.out.println(root.key);
        preorder(root.left);
        preorder(root.right);
    }
}
```

}

```
void levelOrder(Node root) {
    if(root == null) return;
    Queue<Node> q = new Queue<Node>();
    q.enqueue(root);
    while(!q.isEmpty()) {
        Node current = q.poll();
        q.dequeue();
        System.out.println(current.key);
        if(current.left) q.enqueue(current.left);
        if(current.right) q.enqueue(current.right);
    }
}
```

Benimse

Small ifade

$T(n) = T(k) + T(n-k-1) + c$

En iyi durum:

$T(n) = T(n/2) + T(n/2) + c$

En kötü durum:

$T(n) = T(n) + T(n-1) + c$

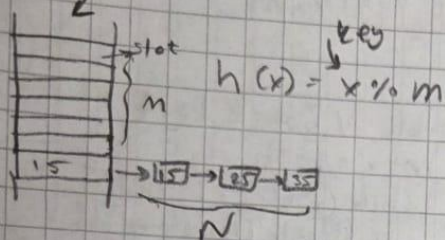
Hashing

Hashtable

key, value

hash function

$h(x)$ Hash Table



$$\alpha = \frac{N}{m}$$

→ open hashing (separate chaining)
closed hashing (open addressing)

public class OpenHashing <K, V> {

private static class Node <K, V> {

K key; V value;

Node <K, V> next;

Node (K key, V value) {

this.key = key;

this.value = value;

}

}

private final int m

private final Node <K, V>[] table;

public OpenHashing (int m) {

this.m = m;

table = new Node [m];

}

private int hash (K key) {

return (key.hashCode() & 0x7fffffff) % m;

}

public void put (K key, V value) {

int i = hash (key);

for (Node <K, V> n = table [i]; n != null; n = n.next) {

if (n.key.equals (key)) {

n.value = value;

return;

}

Node <K, V> node = new Node <> (key, value);

node.next = table [i];

table [i] = node;

}

public V get (K key) {

for (Node <K, V> n = table [hash (key)]; n != null; n = n.next) {

if (n.key.equals (key)) {

return n.value;

return null;

}

public void delete (K key) {

int i = hash (key);

if (table [i] == null)

return;

if (table [i].key.equals (key)) {

table [i] = table [i].next;

return;

}

for (Node <K, V> n = table [i];

n.next != null; n = n.next)

if (n.next.key.equals (key)) {

n.next = n.next.next;

return;

}

}

}

→ closed hashing

Linear probing $\rightarrow p(k, i) = (h(k) + i \cdot c) \% m$
ottoma
sabit
0'den başlar

Quadratic probing $\rightarrow p(k, i) = (h(k) + (i^2 + i + \frac{i}{2}) / 2) \% m$

Double hashing $\rightarrow p(k, i) = (h_1(k) + i \cdot h_2(k)) \% m$

```
public class LinearProbing<K,V>{  
    private static class Entry<K,V>{  
        K key; V value; boolean deleted;  
        Entry(K key, V value){  
            this.key = key;  
            this.value = value;  
        }  
    }  
    private final int m;  
    private Entry<K,V>[] table;  
    private int N;  
  
    public int hash(K key){  
        return (key.hashCode() &  
            0x7fffffff) % m;  
    }  
  
    public boolean put(K key, V value){  
        if (N == m) return false;  
        int i = hash(key);  
        while (table[i] != null && !table[i].deleted){  
            if (table[i].key.equals(key)){  
                table[i].value = value;  
                return true;  
            }  
            i = (i+1) % m;  
        }  
        table[i] = new Entry<>(key, value);  
        N++;  
        return true;  
    }  
}
```

```
    public V get(K key){  
        int i = hash(key);  
        while (table[i] != null){  
            if (!table[i].deleted &&  
                table[i].key.equals(key)){  
                return table[i].value;  
            }  
            i = (i+1) % m;  
        }  
        return null;  
    }  
}
```

```
    public boolean delete(K key){  
        int i = hash(key);  
        while (table[i] != null){  
            if (!table[i].deleted &&  
                table[i].key.equals(key)){  
                table[i].deleted = true;  
                N--;  
                i = (i+1) % m;  
            }  
        }  
        return false;  
    }  
}
```


$$\alpha = \frac{N}{m}$$

$$\binom{N}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{N-k}$$

$$\binom{N}{k} \left(\frac{\alpha}{N}\right)^k \left(1 - \frac{\alpha}{N}\right)^{N-k}$$

$$\binom{N}{k} \left(\frac{\alpha}{N}\right)^k = \frac{N!}{k!(N-k)!} \cdot \frac{\alpha^k}{N^k}$$

$$= \frac{\alpha^k}{k!} \cdot \frac{N(N-1)\dots(N-k+1)}{N^k}$$

$N \rightarrow \infty$

eklenen ve başarısız arama
ortalama sayısı

$$E[\text{karşılaştırma}] = \sum_{k=0}^{\infty} p_k \cdot k = \sum_{k=0}^{\infty} \frac{e^{-\alpha} \cdot \alpha^k}{k!} \cdot k$$

$$= \sum_{k=1}^{\infty} \frac{e^{-\alpha} \cdot \alpha^k}{k!} \cdot k = e^{-\alpha} \sum_{k=1}^{\infty} \frac{\alpha^k \cdot k}{k!} = e^{-\alpha} \sum_{k=1}^{\infty} \frac{\alpha^k}{(k-1)!}$$

$$= e^{-\alpha} \sum_{j=0}^{\infty} \frac{\alpha^{j+1}}{j!} = e^{-\alpha} \sum_{j=0}^{\infty} \frac{\alpha^j \cdot \alpha}{j!} = e^{-\alpha} \cdot \alpha \cdot \sum_{j=0}^{\infty} \frac{\alpha^j}{j!} = e^{-\alpha} \cdot \alpha \cdot e^{\alpha} = \alpha$$

e^x 'nin Taylor serisi

Başarılı arama

$$\left\lfloor \frac{N-1}{m} \right\rfloor + \left(1 + \frac{N-1}{2m}\right) \approx 1 + \frac{\alpha}{2}$$

$$E[\text{parisyon}] = \frac{1}{k} \sum_{i=1}^k i = \frac{k+1}{2}$$

$$E[\text{karşılaştırma}] = \sum_{k=0}^{\infty} p_k \cdot \left(1 + \frac{k-1}{2}\right)$$

$$= \sum_{k=1}^{\infty} p_k \left(\frac{k+1}{2}\right) = \frac{1}{2} \sum_{k=1}^{\infty} p_k (k+1)$$

$$= \frac{1}{2} \left[\sum_{k=1}^{\infty} p_k k + \sum_{k=1}^{\infty} p_k \right] = \frac{1}{2} \left[\alpha + 1 - p_0 \right]$$

$$= \frac{1}{2} \left[\alpha + 1 - e^{-\alpha} \right] = \frac{1 + \alpha - e^{-\alpha}}{2}$$

Bir ekleme veya başarısız arama

$$\alpha = \frac{N}{m}$$

$$\underbrace{\frac{N}{m} \cdot \frac{(N-1)}{(m-1)} \cdot \frac{(N-2)}{(m-2)} \cdots \frac{(N-i+1)}{(m-i+1)}}_{i \text{ tane}} \approx \left(\frac{N}{m}\right)^i = \alpha^i \rightarrow \text{Çok küçük}$$

N ve m
çok büyük.

$$\frac{(m-i) - (N-i)}{(m-i)} = \frac{m-i-N+i}{m-i} = \frac{m-N}{m-i} \approx \frac{m-N}{m} = \frac{1-\alpha}{1} \rightarrow \text{boş slot}$$

$$E[\text{prob}] = \sum_{i=0}^{\infty} (i+1) \cdot \alpha^i \cdot (1-\alpha)$$

$$= (1-\alpha) \cdot \sum_{i=0}^{\infty} (i+1) \cdot \alpha^i$$

$$= (1-\alpha) \cdot \frac{1}{(1-\alpha)^2} = \frac{1}{1-\alpha}$$

Başarılı arama veya silmenin maliyeti

$$= \frac{1}{N} \sum_{i=0}^{N-1} \frac{1}{1-\alpha} = \frac{1}{N} \sum_{i=0}^{N-1} \frac{1}{1-\frac{i}{m}} = \frac{1}{N} \sum_{i=0}^{N-1} \frac{m}{m-i}$$

$$\begin{aligned} x &= i/m \\ dx &= 1/m \\ \alpha &= N/m \end{aligned} \quad \frac{1}{\alpha} \int_0^{\alpha} \frac{1}{1-x} dx = \frac{1}{\alpha} \cdot \left[-\ln(1-x) \right]_0^{\alpha} = \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right)$$

Linear sondalama altında ekleme veya başarısız arama:

$$\frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right)$$

Silme veya başarılı arama:

$$\frac{1}{2} \left(1 + \frac{1}{(1-\alpha)} \right)$$

Detaylı Analiz için Kaynak

Knuth'un "The Art of Computer Programming"

Balanced Tree (B-tree)

Özellikleri

- + Her düğüm birden fazla anahtar barındırabilir.
- + Yapraklar aynı seviyededir.
- + Tüm veri sıralı tutulur.
- + Arama, ekleme ve silme işlemleri $O(\log n)$ karmaşıklığındadır.

B-tree'nin derecesi: $t \geq 2$

Özellik

t neyi ifade eder?	t (minimum degree) minimum çocuk sayısı	t (maximum degree) maksimum çocuk sayısı
maksimum anahtar	$2t-1$	$t-1$
minimum anahtar	$t-1$	$(t/2)-1$
maksimum çocuk	$2t$	t
minimum çocuk	t	$(t/2)$

78 52 81 40 33 80 85 20 38 $t=3$ (minimum degree)

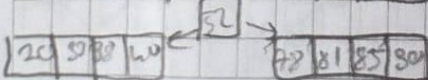
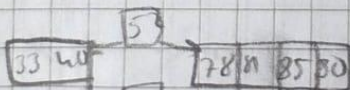
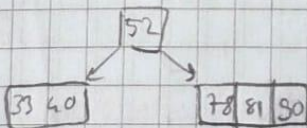
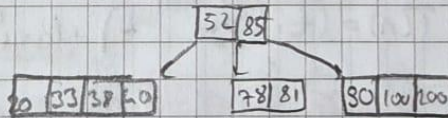
78

52 78

52 78 81

40 52 78 81

33 40 52 78 81



Eklemelerin ardışık toplamı

$$n_{\min} = 1(t-1) + t(t-1) + t^2(t-1) + \dots + t^d(t-1) = \sum_{i=0}^d t^i(t-1)$$

↑
Minimum
ardışık
sayısı

$$= (t-1) \sum_{i=0}^d t^i = (t-1) \cdot \frac{t^{(d+1)} - 1}{(t-1)}$$

$$n_{\min} = t^{(d+1)} - 1$$

$$n = t^{(d+1)} - 1 \Rightarrow \log_t t^{(d+1)} = \log_t (n+1) \Rightarrow d+1 = \log_t (n+1)$$

$$d = \log_t (n+1) - 1$$

$O(h) \rightarrow$ yapraklar eklemeleri

$O(t) \rightarrow$ bölünme

$$\begin{aligned} O(t \cdot h + h) \\ &= O(h(t+1)) \\ &= O(h \cdot t) \\ &\quad \Downarrow \\ &O(h) \end{aligned}$$

$$O(\log_t (n+1) - 1)$$

$$O(\log n)$$

$$T(n) = (t \cdot (\log_t (n+1) - 1) + \log_t (n+1) - 1)$$

$$t \in \{2, 4\}$$

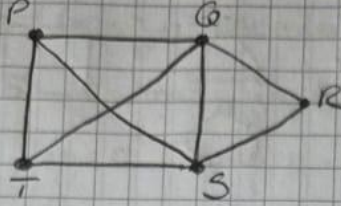
$$n = t^0 - 1$$

Grady Özellikleri

$$G = (V, E)$$

$$E \subseteq V \times V$$

$$|E| \leq |V|^2$$



$$V = \{P, Q, R, S, T\}$$

$$E = \{\{P, Q\}, \{P, T\}, \{P, S\}, \{Q, R\}, \{Q, S\}, \{T, S\}, \{S, R\}\}$$

$$\deg(P) = 3, \deg(Q) = 4$$

$$\deg(R) = 2, \deg(S) = 4, \deg(T) = 3$$

Önemli (Trivial) Graflar

(A)

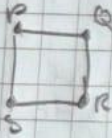
Bos Graflar (Null)

(A)

(B)

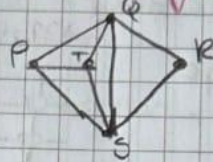
(C)

Yönsüz Graflar



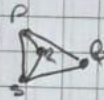
- bağlantıların yönü yoktur.

Basit Graflar



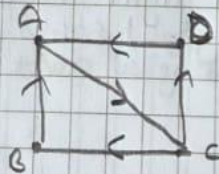
- öz döngü yok. (self-loop).
- çoklu kenar yok.

Düzlemsel Graflar



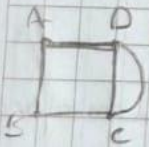
- kenarlar birbirini kesmez.

Yönlü Graflar



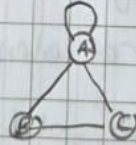
- bağlantıların yönü vardır.

Coklu Graflar



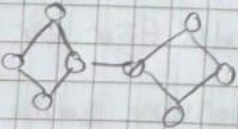
- iki node arasında birden fazla bağlantı olabilir.

Self Graflar

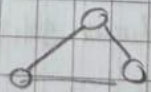


- öz döngü vardır.

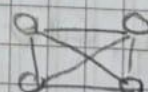
Basit Graflar



Düzenli graflar



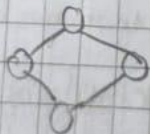
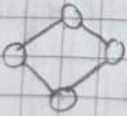
$$\deg = 2$$



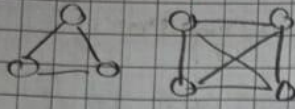
$$\deg = 3$$

- tüm düğümlerin derecesi eşittir.

Bağılantısız Graflar

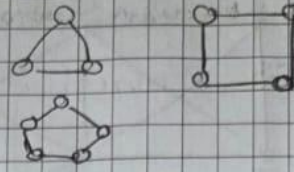


Tem Graf lar



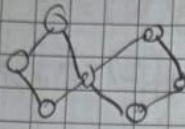
- her köşe 2+1 arasında 1 kenar vardır

(Graf lar or Cycle) Döngü ve Dairesel Graf lar

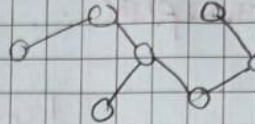


- en az 3 köşeli, her köşenin derecesi 2, bir zincir içinde bağlıdır

Döngüsel Graf lar (Cyclic)



Geçimsiz Graf lar

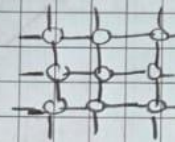


Sonlu Graf lar



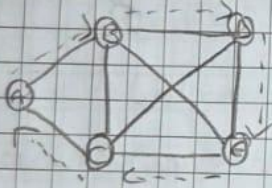
- sonlu sayıda köşe ve kenar

Sonsuz Graf lar



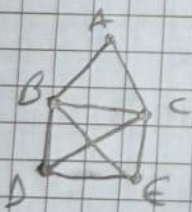
- sonsuz sayıda köşe ve kenar

Hamilton Graf ı



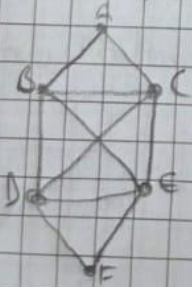
- Her köşeyi tam olarak 1 kez ziyaret eden ve başladığı aynı köşede biten (kapalı bir yola (yeni döngüye) sahip bir graftır. Bu kapalı yol, Hamilton Döngüsü olarak adlandırılır.

Euler Graf ı



- Bir Euler yolu, sonlu bir grafta her kenarı tam olarak 1 kez ziyaret eden ve köşelerin yeniden ziyaret edilmesine izin veren bir yoldur. Benzer şekilde Bir Euler döngüsü, aynı köşelerle başlayıp biten bir Euler yoludur. Eğer bir graf, Euler yoluna sahip ama Euler döngüsüne sahip değilse yarı Euler graf ı olarak adlandırılır. Bunlar ilk kez 1736 yılında Leonhard Euler tarafından meşhur Königsberg Yedi Köprüsü problemi çözülürken tartışılmıştır.

Euler path = D, E, B, C, A, B, D, C, E



Euler Circuit = D E B C A B D C E F D

Derinliğine Öncelikli Arama

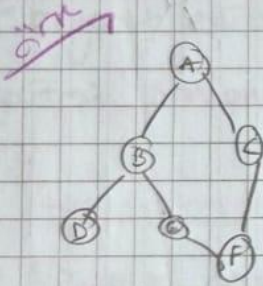
DFS, her zaman bulunduğun düğümden mümkün olduğunca derine gitmeye çalışır.

Standart bir DFS uygulaması, grafin her kısmını iki kategoriden birine koyar:

Ziyaret Edilen, ziyaret Edilmeyen.

İteratif DFS Algoritması aşağıdaki adımlarda çalışır.

- 1) Grafin köşelerinden herhangi birini bir yığınun üstüne koyarak başla.
- 2) Yığın en üst öğesini al ve ziyaret edilenler listesine ekle.
- 3) Bu köşenin bitişik düğümlerinin bir listesini düşün. Ziyaret edilenler listesinde olmayanları yığın en üstüne ekle.
- 4) Yığın boşalana kadar 2. ve 3. adımları tekrarlama devam et.




```

public class DFS {
    static final int MAX_NODES = 100;
    static int[][] graph = new int[MAX_NODES][MAX_NODES];
    static int numNeighbors = new int[MAX_NODES];
    static boolean[] visited = new boolean[MAX_NODES];

    static void addEdge(int u, int v) {
        graph[u][numNeighbors[u]++] = v;
        // graph[u][numNeighbors[u]++] = v -> jante
    }

    static void resetVisited(int numNodes) {
        for (int i = 0; i < numNodes; i++)
            visited[i] = false;
    }

    static void recursive DFS(int node) {
        visited[node] = true;
        System.out.println("Visited " + node + " (recursive)");
        for (int i = 0; i < numNeighbors[node]; i++) {
            int neighbor = graph[node][i];
            if (!visited[neighbor])
                recursive DFS(neighbor);
        }
    }

    static void iterative DFS(int start) {
        Stack<Integer> stack = new Stack<>();
        stack.push(start);
        while (!stack.isEmpty()) {
            int current = stack.pop();
            if (!visited[current]) {
                visited[current] = true;
                System.out.println("Visited " + current + " (iterative)");
                for (int i = numNeighbors[current] - 1; i >= 0; i--) {
                    int neighbor = graph[current][i];
                    if (!visited[neighbor])
                        stack.push(neighbor);
                }
            }
        }
    }

    static void main() {
        int numNodes = 5;
        addEdge(0, 1);
        addEdge(0, 2);
        addEdge(1, 3);
        addEdge(1, 4);
    }
}

```

$O(V)$ $\rightarrow$ Belle Kormasizliji.

$O(V+E)$ $\rightarrow$ Famen Kormasizliji.

Genişliğine Öncelikli Arama (Breadth-First Search)

Algoritmanın çalışma adımları:

- 1) Başlangıç düğümüne Kuyruğa ekle ve işaretli olarak işle.
- 2) Kuyruk boş değilse:
 - a) Kuyruğun başındaki düğümü çıkar.
 - b) Düğümün etrafına yollar veya listeye ekle.
 - c) Tüm işlemler bitene kadar Kuyruğa ekle ve işle.
- 3) Tüm düğümler işlenene kadar devam et.

Dijkstra Zaman karmaşıklığı:

1 extractPriority $\rightarrow \Theta(\log V)$

2 insertPriority $\rightarrow \Theta(\log V)$

V tane için $\rightarrow \Theta(V \log V)$

E tane için $\rightarrow \Theta(E \log V)$

Toplam Priority $\rightarrow \Theta(\log V (E + V))$

Fibonacci heap kullanımı $\rightarrow$ Fibonacci heap Priority $\Theta(1)$

Toplam Priority $\Theta(E + V)$

Bellek karmaşıklığı:

$\Theta(V)$

Bellman-Ford Algoritması $\rightarrow$ Dinamik prog.

Çalışma adımları:

- 1-Kaynak köşe için başlangıç mesafesini 0'a ayarla. Diğer tüm köşeler için mesafeleri ∞ 'a ayarla.
- 2-Döngüsüz en uzun kısa yol en fazla $V-1$ kenara sahip olduğu için tüm kenarları $V-1$ kez kontrol et (Kontrol işlemi için bitişliklik matrisi [Adjacency matrix] kullanılır) Her kenar için daha kısa bir mesafenin hesaplanıp hesaplanmayacağını kontrol et ve hesaplanan mesafe daha kısaysa mesafeyi güncelle.
- 3-1 tur daha tara. Eğer hala güncelleme oluyorsa grafta negatif döngü vardır demektir. Bu durumda algoritma doğru sonuç üretmez.

avantajları:

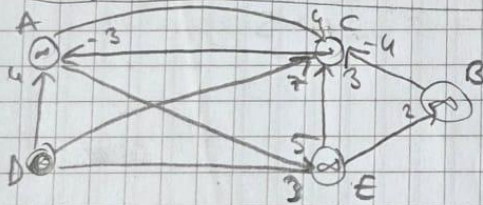
1 Negatif dngülerin olmadığı negatif ağırlıklı yönlü grafları ele alabilir.

2 Yönlü ve yönsüz negatif olmayan ağırlıklı graflar için acaılır.

3 Negatif dngülerin varlığını kontrol etmek için kullanılabilir.

negatif dngü kontrolü

```
for(int j=0; j<E; j++) {
    int u = graph.edges[j].src;
    int v = graph.edges[j].dest;
    int w = graph.edges[j].weight;
    if (dist[u] != INF & & dist[u] + w < dist[v])
        System.out.println("tespit edildi");
}
```



iterasyon	A	B	C	D	E
başlangıç	∞	∞	∞	0	∞
1.	4	5	1	0	3
2	-2	5	1	0	3
3					
4					

$V = \{(D,A), (D,C), (D,E), (A,E), (A,C), (C,A), (C,E), (E,B), (B,C)\}$ $|V|-1$ kez

Dezavantajları:

1 Dijkstra algoritmasından daha yüksek zaman karmaşıklığına sahiptir. Bu nedenle uygulama alanı Dijkstra'ya kıyasla daha azdır.

```
for(int i=1; i<=V; i++) {
    for(int j=0; j<E; j++) {
        int u = graph.edges[j].src;
        int v = graph.edges[j].dest;
        int w = graph.edges[j].weight;
        if (dist[u] != INF & &
            dist[u] + w < dist[v])
            dist[v] = dist[u] + w;
    }
}
```

zaman karmaşıklığı:

$$O(E \cdot |V|)$$

Ekstra karmaşıklık:

$$O(|V|)$$

Floyd Warshall Algoritması

- 1 Her düğümden kendisine mesafe 0'dır. Doğrudan bağlı düğümler için kenar ağırlığını yaz. Bağlantı yoksa mesafe ∞ kabul edilir.
- 2 Her düğümü sırayla ara durak olarak seç. Ara durak üzerinden geçen tüm $i \rightarrow j$ çiftlerini kontrol et ve mesajları güncelle. Her yeni ara durak bir önceki iterasyonda bulunan kısa yolların üzerine inşa edilir. Böylece birden fazla ara duraklı yollar da parça parça keşfedilir. Bu tüm düğümler için tekrarlar:
- 3 $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ ise $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$
- 3 Eğer bir düğümün kendisine mesajı negatifse negatif döngü vardır. Çünkü bir düğümün kendisine mesafesi normalde 0'dır.

avantajları

Bir graftaki düğümler arasındaki en kısa yolu bulmanın dışında bir grafin geçişli kapanmasını (transitive closure) hesaplamak ve bir graftaki negatif döngüleri tespit etmek için de kullanılır.

bellek $\rightarrow V^2$

kod $\rightarrow$ sadece 3 döngü
 matris S

dezavantajları

Bellek karmaşıklığı yüksektir

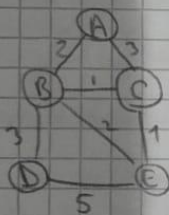
Yönsüz

Minimum Maliyetli Ağaçlar (Spanning Tree)

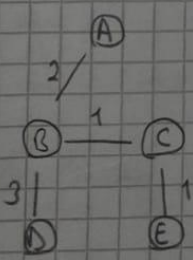
- 1) Grafin tüm düğümlerini içerir
- 2) Tüm düğümler $V-1$ kenar içerir
- 3) Denge içermes

Kruskal Algoritması

- 1) Grafin tüm kenarları maliyete göre küçükten büyüğe doğru sıralanır.
- 2) Grafin kullanılmayan en düşük maliyetli kenar seçilir.
- 3) Seçilen kenar bir denge oluşturuyorsa eklenir.
- 4) Ağaç bir ağaç olana kadar 2. ve 3. adımları tekrarlanır.



B-C	1 ✓
C-E	1 ✓
B-E	2 ✗
A-B	2 ✓
B-D	3 ✓
A-C	3 ✗
D-E	5 ✗



```
static int[] parent;
static int find(int i) {
    if (parent[i] == i)
        return i;
    return parent[i] = find(parent[i]);
}
```

```
static void unionSet(int x, int y) {
    int xroot = find(x);
    int yroot = find(y);
    parent[xroot] = yroot;
}
```

```
static void KruskalMST(int V, Edge[] edges) {
    Edge[] result = new Edge[V];
    int e = 0;
    parent = new int[V];
    for (int v = 0; v < V; v++)
        parent[v] = v;
    Arrays.sort(edges, Comparator.comparingInt(edge -> edge.weight));
```

```
int i = 0;
while (e < V-1 && i < edges.length) {
    Edge next = edges[i++];
    int x = find(next.src);
    int y = find(next.dest);
    if (x != y) { // Denge oluşturulmazken kenar eklenir
        result[e++] = next;
        unionSet(x, y); // union-find kullanılır
    }
}
```

IEI

3

$|E| \leq |V|$

Sort maliyeti $\rightarrow |E| \cdot \log |E|$

$T(|E|, |V|) = |E| \cdot \log |E| + |E| \cdot \log |V|$

Ödev $\rightarrow$ Tüm büyük testler için union-find da gereksizdir.

$= |E| \log |E| + |E|$

$= \Theta(|E| \log |E|)$

$\Theta(|E| \cdot \log |V|) \rightarrow \Theta(|E|^2 \cdot \log |V|)$

$\Theta(|E| \cdot \log |V|)$

Ödev $\rightarrow$ Prim Algoritması

$\rightarrow$ Boruvka Algoritması

$\rightarrow$ Dinic Programları

$\rightarrow$ Grid Yolları

Avantajları

- 1) Anlaşılması nispeten kolay
- 2) MST düzeltilmesi garanti eder
- 3) Başlı alternatifleri parçalayabilir.
- 4) Bağlantıları bileşenleri sahip grafları içerebilir.
- 5) Diğer graflar için ayrı ayrı MST bulabilir.

Dezavantajı

- 1) Yoğun graflar için yavaş çalışır.
- 2) Dinamik graf genişletmelerini desteklenmez.

Hesaplama Sınırları

Ös:

A. B

↳ nym

[0
A]

Karar Pro
(Decision Pro

Traveling
aralardaki
bir turla

Knapsack
sınırlı. Bun
görüldüğü re

Hesaplamanın Sınırları

Arama algoritmaları en kötü durum $\log n$

Sıralama algoritmaları en kötü durum $n \log n$

Dijkstra $O[(|E|+|V|)\log|V|]$

Floyd-Warshall $O(|V|^3)$

Bellman Ford $O(|V| \cdot |E|)$

Strassen'in hızlı matris çarpımı $O(n^{2.81})$

İndirgeme (Reduction)

örn 1

Elimizde X ve Y gibi iki dizi olsun X 'deki en küçük değeri Y 'deki en küçük değeri, X 'deki bir sonraki küçük değeri Y 'deki bir sonraki en küçük değeri eşleştireceği ve bu şekilde devam eden iki dizideki öğelerin eşleşmesi problemi ele alalım.

$A \leq_p B$ ve $T_B(n) \in O(f(n))$ ise $T_A(n) \in O(f(n))$

$A \leq_p B$ ve $T_A(n) \in \Omega(f(n))$ ise $T_B(n) \in \Omega(f(n))$

A problemi
polinom
zamanında
B problemine
indirgenir.

örn 2 n basamaklı iki sayının çarpımı problemi ele alalım.

$\begin{array}{r} 12 \\ \times 34 \\ \hline \end{array}$

n^2 'lik
çarpma
mekanizması $O(n^2)$

$\Rightarrow$

Hızlı çarpma
Alg.

Karatuba
(böl ve çarp)

örn 3 $n \times n$ 'lik iki matrisin çarpımı problemi ele alalım.

A, B

$\rightarrow n \times n$

$\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}$
 $\begin{bmatrix} 0 & B^T \\ B & 0 \end{bmatrix}$

$\Rightarrow 2n \times 2n$

$\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}$

$\begin{bmatrix} 0 & B^T \\ B & 0 \end{bmatrix}$

$\Rightarrow \begin{bmatrix} A \cdot B & 0 \\ 0 & A^T \cdot B^T \end{bmatrix}$

Karar Problemi: Cevabı yalnızca "evet" ya da "hayır" olan problemlerdir.

Traveling Salesman problem: Varsayalım 6 şehir var ve aralarındaki mesafeler verildi. Bu şehirleri 300 km'den kısa bir turla gezebilir miyim?

Knapsack problem: Bir sırtçanta var ve kapasitesi sınırlı. Bunun dışında bir dizi eşya var. Her birinin ağırlığı ve değeri belli. Ancak tüm eşyaları alamayız. Maksimum değer elde etmek istiyoruz. Yalnızca ağırlık sınırını aşmadan. Buna göre bu eşyalar arasından dyle bir seçim yapabilir miyim? Toplam ağırlık 15 kg'ı geçmesin toplam değer en az 100 olsun.

3-Satisfiability Problem: $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_4) \wedge (x_2 \vee x_3 \vee \neg x_5)$
(3-SAT)
Tüm bu OR'larda bağlanmış 3'lü cümleciklerin hepsi aynı anda TRUE dur mu? Diğer bir deyişle bu formülü TRUE yapacak bir değişken ataması var mı?

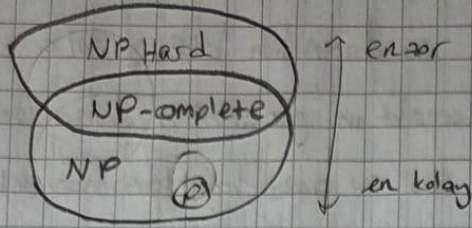
Optimizasyon Problemi: Olası çözümler kümesi üzerinde bir amaç fonksiyonunun en iyi değerini (minimum veya maksimum) arandığı problemdir.

TS Prob: ... tüm şehirleri kapsayan en kısa tur kaç km'dir.

Knapsack Prob: ... bu çanta kapasitesiyle alabileceğim maksimum değer nedir?

Max 3-SAT Prob: ... OR'larda bağlanmış 3'lü cümleciklerin kaç TRUE yapılabilir?

Polinom zaman Problemleri: Polinom zamanda çözülebilen problemlerdir.



Deterministik Olmayan Polinom Zaman Problemleri:

- Deterministik olmayan bir Turing makinesi ile polinom zamanda çözülebilen problemlerdir.
- Problemin çözüm veya çözümlerinin doğrulanması polinom zamanda Turing makinesi ile yapılabilen problemlerdir.
- Non-deterministik bir Turing makinesinin her adımında mümkün olan tüm hesaplama dallarını takip ettiği ve bu dallardan en az birini kabul durumuna ulaşırsa makinenin bunu kabul ettiğini hatırlar.

Örnek: SAT, Hamiltonian path, Knapsack, Vertex cover problem.

NP-Complete Problemler:

- Problem NP'de olmalıdır. Yani çözümü polinom zamanda doğrulanabilirliktir.
- NP'deki her problem, bu probleme polinom zamanda indirgenebilirliktir.

Örnek: TS problem, Knapsack, Br SAT, Graf renklendirme, bir grafın Hamilton düğümüne sahip olup olmadığını belirleme.

NP-Hard Problemler:

NP'deki her problemin polinom zamanda indirgenebileceği problemlerdir.

NP-Hard problemler NP'de olmak zorunda değildir. Dolayısıyla çözümü polinom zamanda doğrulanamazdır.

$P=NP$ mi?

Bilgisayar biliminin en büyük açık sorularından biri şudur:
Bir problemi çözmek, çözümü doğrulamaktan gerçekten daha zor
mudur? Şezgilerimiz evet der. Ancak bu şu anda kanıtlanamamaktadır.
1871'den beri yanıtlanamayan bu soru, matematik ve bilgisayar
biliminin seyrini değiştirebilecek güçtedir.

⋮

Scott Aaronson $\Rightarrow$ <https://scottaaronson.blog/?p=122>

⋮

yani $P=NP$ olsaydı, bir problemi çözmek ile çözümü doğrulamak
arasındaki fark ortadan kalkardı.